```
  length: number; #A
}
function loggingIdentity<T extends Lengthwise>(arg: T): T {
  console.log(arg.length); #B
  return arg;
}
```

A generic can be constrained to a particular class or interface, ensuring only compatible types are used. In listing 3.18, T must be a subtype of `Animal`, so it must have both `name` and `age` properties.

Listing 3.18 Class/interface constraint

```
interface Animal {
  name: string;
  age: number;
}
function getOldest<T extends Animal>(animals: T[]): T {
  if (animals.length === 0) {
    throw new Error("Cannot get oldest of empty array");
  }
  return animals.reduce((oldest, current) => #A
    current.age > oldest.age ? current : oldest
  );
}
```

`keyof` constraint ensures that a generic type parameter is always a valid property name (key) of another object type. In listing 3.19, `<T, K extends keyof T>` guarantees that K must be a key of T.

Listing 3.19 Keyof constraint

```
function getProperty<T, K extends keyof T>(obj: T, key: K): T[K]
  return obj[key];
}
```

Calling `getProperty({ a: 1, b: 2 }, "a")` succeeds because "a" is a valid key of the object. However, attempting `getProperty({ a: 1, b: 2 }, "c")` causes TypeScript to raise a compile-time error since "c" is not a key of the provided object. This compile-time check eliminates the risk of runtime errors when accessing object properties, making code more reliable.